

Introdução a Classes e Objetos

Linguagem de Programação II

Universidade Estadual da Paraíba
Curso: Ciência da Computação

September 28, 2026

O que vamos aprender hoje?

- 1 O problema de ter **muitas variáveis** separadas
- 2 A ideia de **agrupar** dados relacionados
- 3 Como criar nossa primeira **classe** em Java
- 4 O que é um **tipo de dado** e tipagem **estática** x **dinâmica**
- 5 A diferença entre tipos **primitivos** e tipos **objeto**
- 6 Por que uma classe cria um **tipo novo**
- 7 Como criar e usar **objetos**
- 8 Vários exemplos de classes bem simples

Por enquanto, nossas classes terão só **atributos** – métodos e construtores vêm nas próximas aulas.

Sumário

- 1 O Problema de Muitas Variáveis
- 2 Agrupando Dados Relacionados
- 3 Criando Nossa Primeira Classe
- 4 Uma Classe Cria um Tipo Novo
- 5 Criando e Usando um Objeto
- 6 O Poder de Ter uma Classe
- 7 Mais Exemplos de Classes

- 1 O Problema de Muitas Variáveis
- 2 Agrupando Dados Relacionados
- 3 Criando Nossa Primeira Classe
- 4 Uma Classe Cria um Tipo Novo
- 5 Criando e Usando um Objeto
- 6 O Poder de Ter uma Classe
- 7 Mais Exemplos de Classes

Guardando os Dados de um Aluno

Imagine que precisamos guardar o nome, a idade e a nota de um aluno:

```
1 String nomeAluno1 = "Maria";  
2 int idadeAluno1 = 20;  
3 double notaAluno1 = 8.5;
```

Até aqui, tudo bem.

E se Forem Dois Alunos?

```
1 String nomeAluno1 = "Maria";  
2 int idadeAluno1 = 20;  
3 double notaAluno1 = 8.5;  
4  
5 String nomeAluno2 = "Joao";  
6 int idadeAluno2 = 22;  
7 double notaAluno2 = 7.0;
```

Já são **6** variáveis para lembrar!

E se Forem 30 Alunos?

```
nomeAluno1, idadeAluno1, notaAluno1  
nomeAluno2, idadeAluno2, notaAluno2  
nomeAluno3, idadeAluno3, notaAluno3  
...  
nomeAluno30, idadeAluno30, notaAluno30
```

90 variáveis!

Por que Isso é um Problema?

- Fácil de **perder o controle** de qual variável pertence a qual aluno;
- Fácil de **errar**: trocar `notaAluno2` por `notaAluno3` sem perceber;
- Imagine escrever um método que precisa receber os dados de **um** aluno – seriam 3 parâmetros, só para um aluno!

Já Sabemos uma Solução Parcial: Vetores

- Já aprendemos vetores – por que não usar um vetor para cada informação?

```
1 String[] nomes = new String[30];  
2 int[] idades = new int[30];  
3 double[] notas = new double[30];
```

Visualizando os Vetores Paralelos

nomes	Maria	Joao	Ana
idades	20	22	19
notas	8.5	7.0	9.2

O aluno de índice 1 é "Joao", 22 anos, nota 7.0 – **três vetores diferentes**, sincronizados só pelo índice.

O Problema dos Vetores Paralelos

☹ Erro Comum

Se você ordenar o vetor `notas` (por exemplo, da maior para a menor), precisa lembrar de reorganizar `nomes` e `idades` **exatamente na mesma ordem** – senão os dados de cada aluno ficam todos trocados!

Nome, idade e nota de um aluno são **uma coisa só** – mas continuam espalhados em três vetores separados.

♡ Checkpoint

- Muitas variáveis separadas viram um caos rapidamente;
- Vetores paralelos ajudam, mas continuam **separando** dados que deveriam estar juntos;
- Precisamos de um jeito de **agrupar** nome, idade e nota de um mesmo aluno.

Sumário

- 1 O Problema de Muitas Variáveis
- 2 Agrupando Dados Relacionados
- 3 Criando Nossa Primeira Classe
- 4 Uma Classe Cria um Tipo Novo
- 5 Criando e Usando um Objeto
- 6 O Poder de Ter uma Classe
- 7 Mais Exemplos de Classes

E se pudéssemos juntar nome, idade e nota
num **"pacotinho"** só, representando **um** aluno?

Uma Analogia

Analogia

Pense numa **ficha de cadastro** de papel: um único formulário com campos para "Nome:", "Idade:" e "Nota:". Cada aluno tem a **sua própria** fichinha preenchida – mas todas as fichas seguem o **mesmo modelo**.

Em Outras Linguagens: o struct

Linguagens como C resolvem isso com algo chamado struct – uma forma de agrupar variáveis relacionadas:

```
1 struct Aluno {  
2     string nome;  
3     int idade;  
4     float nota;  
5 };
```

(isto é só uma ilustração da ideia – não é código Java)

A Ideia Central do struct

- Cria um **novo modelo** chamado `Aluno`, com 3 campos dentro;
- Cada vez que você precisar de "um aluno", usa esse modelo – **um pacote só**, em vez de 3 variáveis soltas.

Java não tem struct – mas tem algo **ainda mais poderoso** para isso: a **classe**.

Por enquanto, vamos usar a classe exatamente como um struct – só para agrupar dados. Mais pra frente, veremos que ela pode muito mais.

♡ Checkpoint

- A ideia é agrupar variáveis relacionadas num "pacote" só;
- Em C, isso se chama `struct`;
- Em Java, vamos usar uma **classe** para fazer o mesmo – e muito mais, no futuro.

Sumário

- 1 O Problema de Muitas Variáveis
- 2 Agrupando Dados Relacionados
- 3 Criando Nossa Primeira Classe**
- 4 Uma Classe Cria um Tipo Novo
- 5 Criando e Usando um Objeto
- 6 O Poder de Ter uma Classe
- 7 Mais Exemplos de Classes

A Classe Aluno

```
1 public class Aluno {  
2     String nome;  
3     int idade;  
4     double nota;  
5 }
```

Repare como é parecido com o struct que vimos!

Quebrando em Pedacos

- `public class Aluno`: já conhecemos – estamos declarando uma classe chamada `Aluno`;
- `String nome;`, `int idade;`, `double nota;`: variáveis declaradas **dentro** da classe, sem estarem dentro de nenhum método.

O que é um Atributo?

😊 Definição: Atributo

É uma variável declarada **dentro** de uma classe (e fora de qualquer método). Representa uma **característica** que todo objeto dessa classe vai ter.

Todo Aluno vai ter um nome, uma idade e uma nota – só que cada um com valores diferentes.

A Classe Sozinha Ainda Não é um Aluno

- A classe `Aluno` é só o **modelo** – a fichinha em branco;
- Ela ainda não representa nenhum aluno de verdade, com nome e nota preenchidos;
- Para isso, precisamos criar um **objeto** – já, já chegamos lá!

Comparando: struct x classe

struct (C)	classe (Java)
<pre>struct Aluno { ... };</pre>	<pre>public class Aluno { ... }</pre>
Agrupar variáveis relacionadas	Agrupar variáveis relacionadas
Só isso, por enquanto	Também pode ter métodos (mais pra frente!)

♡ Checkpoint

- Uma classe pode agrupar vários atributos relacionados;
- Atributo = variável declarada dentro da classe, fora de métodos;
- A classe é o **modelo** – ainda não é um aluno de verdade.

Sumário

- 1 O Problema de Muitas Variáveis
- 2 Agrupando Dados Relacionados
- 3 Criando Nossa Primeira Classe
- 4 Uma Classe Cria um Tipo Novo**
- 5 Criando e Usando um Objeto
- 6 O Poder de Ter uma Classe
- 7 Mais Exemplos de Classes

O que é um Tipo de Dado?

😊 Definição: Tipo de Dado

Um tipo define **que valores** uma variável pode guardar e **que operações** fazem sentido com ela.

😊 Analogia

Pense em gavetas de formatos diferentes: uma gaveta para **números inteiros**, outra para **texto**, outra para **verdadeiro/falso**. Cada gaveta só serve para guardar um tipo de coisa.

Duas Grandes Famílias de Tipos

Em Java, todo tipo pertence a uma de duas famílias:

Primitivo ou **Objeto**

Tipos Primitivos

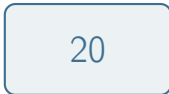
😊 Definição: Tipo Primitivo

É um tipo **básico**, que já vem pronto no Java. A variável primitiva **já** é o valor – não existe nenhuma "caixinha" por dentro.

Só existem 8 tipos primitivos em Java: `int`, `double`, `boolean`, `char`, `long`, `float`, `byte`, `short`.

Como é um Tipo Primitivo?

```
int idade = 20;
```



A variável `idade` **já** é o valor 20 – simples assim, sem nada guardado por dentro.

Tipos Objeto (ou Referência)

😊 Definição: Tipo Objeto

É um tipo criado a partir de uma **classe**. A variável objeto representa uma **caixinha**, e é dentro dela que o(s) dado(s) de verdade moram.

Como é um Tipo Objeto?

```
String nome = "Maria";
```

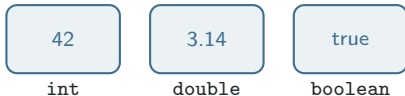
objeto do tipo String



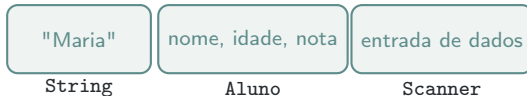
A variável `nome` representa uma **caixinha**. Aqui a caixinha guarda só um texto – mas é uma caixinha de verdade, criada a partir de uma classe.

Galeria de Tipos: Primitivos x Objetos

Primitivos



Objetos



Primitivos guardam **o valor direto**. Objetos são **caixinhas** – podem guardar um valor simples (`String`) ou vários valores juntos (`Aluno`).

Um Detalhe Curioso: a String

😊 Dica

Repare que `String nome = "Maria";` tem a **mesma cara** de `int idade = 20;` – por isso é fácil achar que `String` é primitivo. Mas **não é!** `String` é uma **classe** do próprio Java, e toda variável `String` é, na verdade, um **objeto**.

Primitivo x Objeto

	Primitivo	Objeto
Guarda	o valor direto	uma caixinha com dado(s) dentro
Nome do tipo	minúscula (int)	Maiúscula (String, Aluno)
Criado com new?	Não	Quase sempre, sim
Valor padrão	0, false...	null (nenhuma caixinha ainda)
Exemplos	int, double, boolean	String, Scanner, Aluno

♡ Checkpoint

- Primitivo = o valor puro, sem caixinha por dentro;
- Objeto = uma caixinha criada a partir de uma classe, que guarda um ou vários dados;
- String parece primitivo na sintaxe, mas é objeto – é uma classe do Java.

Tipos que Já Conhecemos

Tipo	Guarda	Categoria
int	números inteiros (sem casas decimais)	Primitivo
double	números com casas decimais	Primitivo
boolean	verdadeiro ou falso	Primitivo
String	texto	Objeto

Tipagem Estática x Dinâmica

😊 Definição: Tipagem Estática

O tipo de cada variável é definido **antes** do programa rodar, e o compilador confere se está tudo certo. Java funciona assim.

😊 Definição: Tipagem Dinâmica

O tipo só é descoberto **durante** a execução do programa. Linguagens como Python e JavaScript funcionam assim.

Comparando na Prática

```
1 // Java -- tipagem estatica
2 int idade = 20;
3 idade = "vinte"; // ERRO! o compilador acusa antes de rodar
```

```
1 # Python -- tipagem dinamica
2 idade = 20
3 idade = "vinte" # funciona, o tipo muda em tempo de execucao
```

Por que Isso Importa?

Dica

Em Java, se você errar o tipo, o **compilador avisa antes** de o programa rodar. Isso ajuda a pegar erros cedo – é um dos motivos de Java ser tão usado em sistemas grandes.

Não Confunda os Dois Assuntos

😊 Dica

Primitivo x Objeto é sobre **o que o tipo guarda** (valor puro ou caixinha). **Tipagem estática x dinâmica** é sobre **quando** o tipo é conferido (antes ou durante a execução). São duas perguntas diferentes sobre o mesmo assunto: tipos.

♡ Checkpoint

- Tipo de dado define o que a variável guarda e o que se pode fazer com ela;
- Java tem tipagem **estática**: o tipo é verificado antes do programa rodar;
- Isso ajuda a evitar erros – o compilador avisa antes da hora.

Relembrando: o que é um Tipo?

- Já usamos vários tipos: `int`, `double`, `boolean`, `String`...
- Um **tipo** define o que uma variável pode guardar, e o que podemos fazer com ela.

Quando criamos "public class Aluno {...}", nós literalmente criamos um TIPO NOVO chamado Aluno!

E como toda classe é usada com `new`, esse tipo novo é sempre um tipo **objeto** – nunca um primitivo. Só existem 8 primitivos fixos em Java; você não pode criar um novo.

Tipos que Já Existiam x Tipos que Criamos

Tipos prontos do Java	Tipos que nós criamos
int, double, boolean	Aluno
String, Scanner	(e outros que ainda vamos criar!)

Lembra que String também é uma classe? Agora Aluno entra nessa mesma "categoria" – e todo tipo que criamos com class é sempre um tipo **objeto**.

Declarando uma Variável do Tipo Aluno

```
1 Aluno meuAluno;
```

Exatamente como fazemos com `int idade;` ou `String nome;` – só que agora o tipo é `Aluno`, um tipo que **nós** criamos.

Dica de Nomenclatura

😊 Dica

Nomes de **classes** (tipos) começam com letra **Maiúscula**: Aluno. Nomes de **variáveis** começam com letra **minúscula**: meuAluno. Essa convenção já apareceu com String – e vale para qualquer classe que criarmos.

Checkpoint

♡ Checkpoint

- Toda classe cria um **tipo novo**, e esse tipo novo é sempre um **objeto**;
- Aluno agora pode ser usado como tipo, igual `int` ou `String`;
- Classe (tipo) começa com Maiúscula; variável começa com minúscula.

Sumário

- 1 O Problema de Muitas Variáveis
- 2 Agrupando Dados Relacionados
- 3 Criando Nossa Primeira Classe
- 4 Uma Classe Cria um Tipo Novo
- 5 Criando e Usando um Objeto**
- 6 O Poder de Ter uma Classe
- 7 Mais Exemplos de Classes

Relembrando o new

- Já usamos new para criar um Scanner: `new Scanner(System.in);`
- new é o comando que **cria de fato** um objeto de uma classe.

Criando um Objeto Aluno

```
1 Aluno meuAluno = new Aluno();
```

- Aluno: o tipo da variável;
- meuAluno: o nome que demos a essa variável;
- new Aluno(): cria, de fato, um **objeto** – um aluno de verdade, ainda sem dados preenchidos.

Valores Padrão

Assim que criado com `new`, cada atributo já começa com um valor padrão:

Tipo do atributo	Valor padrão
<code>int</code> , <code>double</code>	<code>0</code>
<code>boolean</code>	<code>false</code>
<code>String</code> (e outras classes)	<code>null</code> (vazio, "nenhum objeto ainda")

Preenchendo os Atributos

```
1 Aluno meuAluno = new Aluno();  
2 meuAluno.nome = "Maria";  
3 meuAluno.idade = 20;  
4 meuAluno.nota = 8.5;
```

Usamos um **ponto** (.) para acessar um atributo de um objeto: `objeto.atributo`.

Lendo os Atributos

```
1 System.out.println(meuAluno.nome);  
2 System.out.println(meuAluno.idade);  
3 System.out.println(meuAluno.nota);
```

Do mesmo jeito, com o ponto, para **ler** o valor de volta.

Visualizando o Objeto

objeto meuAluno, do tipo Aluno

```
nome: "Maria"
```

```
idade: 20
```

```
nota: 8.5
```

Erro Comum

Declarar a variável mas esquecer o new: `Aluno aluno; aluno.nome = "Maria";` – isso gera um `NullPointerException`, porque `aluno` ainda não aponta para nenhum objeto de verdade. Sem `new`, não existe objeto nenhum!

☹ Erro Comum

Confundir o **tipo** com o **nome da variável**: em `Aluno aluno = new Aluno();`, `Aluno` (Maiúsculo) é o tipo/classe, e `aluno` (minúsculo) é só o nome que escolhemos para a variável – poderia ser qualquer outro nome.

♡ Checkpoint

- `new NomeDaClasse()` cria de fato um objeto;
- Sem `new`, a variável não aponta para nenhum objeto – cuidado com `NullPointerException`;
- Acessamos e alteramos atributos com `objeto.atributo`.

Sumário

- 1 O Problema de Muitas Variáveis
- 2 Agrupando Dados Relacionados
- 3 Criando Nossa Primeira Classe
- 4 Uma Classe Cria um Tipo Novo
- 5 Criando e Usando um Objeto
- 6 O Poder de Ter uma Classe**
- 7 Mais Exemplos de Classes

Criando um Segundo Aluno

```
1 Aluno aluno1 = new Aluno();  
2 aluno1.nome = "Maria";  
3 aluno1.idade = 20;  
4 aluno1.nota = 8.5;  
5  
6 Aluno aluno2 = new Aluno();  
7 aluno2.nome = "Joao";  
8 aluno2.idade = 22;  
9 aluno2.nota = 7.0;
```

Cada Objeto tem Seus Próprios Atributos



Mudar a nota de `aluno1` **não afeta** `aluno2` – são objetos completamente independentes.

Lembra dos Vetores?

- Já sabemos criar vetores de `int`, de `String`...
- Agora que `Aluno` é um **tipo**, podemos ter um vetor de `Aluno` também!

Um Vetor de Objetos

```
1 Aluno[] turma = new Aluno[3];  
2 turma[0] = aluno1;  
3 turma[1] = aluno2;  
4 turma[2] = new Aluno();
```

Um vetor onde cada posição guarda um Aluno **inteiro** – nome, idade e nota juntos.

Lembra das **90 variáveis** separadas, lá no início da aula?

Agora, para 30 alunos, basta:

```
1 Aluno[] turma = new Aluno[30];
```

Uma linha só – organizada, clara, sem risco de misturar os dados de alunos diferentes.

♡ Checkpoint

- Cada objeto guarda seus **próprios** valores de atributo, independente dos outros;
- Podemos ter vetores de objetos, assim como temos de `int` ou `String`;
- Isso resolve, de vez, o problema de organização que vimos no início da aula.

Sumário

- 1 O Problema de Muitas Variáveis
- 2 Agrupando Dados Relacionados
- 3 Criando Nossa Primeira Classe
- 4 Uma Classe Cria um Tipo Novo
- 5 Criando e Usando um Objeto
- 6 O Poder de Ter uma Classe
- 7 Mais Exemplos de Classes**

O Padrão se Repete

O mesmo padrão que usamos para `Aluno` serve para **qualquer** "coisa" do mundo real que queremos representar.

Exemplo: Livro

😊 Exemplo: Classe Livro

```
1 public class Livro {  
2     String titulo;  
3     String autor;  
4     int numeroPaginas;  
5 }
```

```
1 Livro meuLivro = new Livro();  
2 meuLivro.titulo = "Dom Casmurro";  
3 meuLivro.autor = "Machado de Assis";  
4 meuLivro.numeroPaginas = 256;
```

Exemplo: Produto

😊 Exemplo: Classe Produto

```
1 public class Produto {  
2     String nome;  
3     double preco;  
4     int quantidade;  
5 }
```

```
1 Produto produto1 = new Produto();  
2 produto1.nome = "Caderno";  
3 produto1.preco = 12.90;  
4 produto1.quantidade = 50;
```

Exemplo: Ponto

☺ Exemplo: Classe Ponto

```
1 public class Ponto {  
2     double x;  
3     double y;  
4 }
```

```
1 Ponto p = new Ponto();  
2 p.x = 3.5;  
3 p.y = 7.2;
```

Exemplo: Carro

😊 Exemplo: Classe Carro

```
1 public class Carro {  
2     String modelo;  
3     int ano;  
4     String placa;  
5 }
```

```
1 Carro meuCarro = new Carro();  
2 meuCarro.modelo = "Fusca";  
3 meuCarro.ano = 1978;  
4 meuCarro.placa = "ABC1234";
```

♡ Checkpoint

- Aluno, Livro, Produto, Ponto, Carro... – todos seguem o **mesmo padrão**;
- Escolha os atributos que fazem sentido para o que você quer representar;
- Esse é o primeiro passo de toda a Programação Orientada a Objetos.

Resumo da Aula

- 1 Muitas variáveis soltas (ou vetores paralelos) viram um caos difícil de manter;
- 2 Uma **classe** agrupa atributos relacionados, assim como um struct faria em C;
- 3 Toda classe cria um **tipo novo**, que podemos usar como `int` ou `String`;
- 4 Primitivo = o valor puro; Objeto = uma **caixinha** criada a partir de uma classe – `String` parece primitivo, mas é objeto;
- 5 Java tem tipagem **estática**: o compilador confere os tipos antes de rodar;
- 6 `new` cria um **objeto** de verdade; acessamos atributos com `objeto.atributo`;
- 7 Cada objeto guarda seus próprios valores, independente dos outros;
- 8 O mesmo padrão serve para representar **qualquer** coisa: alunos, livros, produtos, pontos, carros. . .